



Moove Education Platform

System & Architecture Design Document (SDD)

Version 3.1
July 21, 2026

Prepared By:
Moove

This product includes software from the Open edX project, licensed under Apache 2.0.
July 28, 2026

Contents

List of Figures	2
List of Tables	3
0.1 Document Management and Quality Assurance	4
0.1.1 Document Purpose	4
0.1.2 Diagram Management	4
0.1.3 Diagram Standards	4
0.1.4 Quality Assurance	4
0.2 System Overview	4
0.3 Physical Architecture	5
0.4 Logical Architecture	5
0.4.1 Presentation Layer	5
0.4.2 Business Logic Layer	5
0.4.3 Data Layer	5
0.5 System Architecture and Data Flow	5
0.5.1 Key Components	5
0.5.2 System Architecture Diagram	6
0.5.3 Data Flow Diagram (DFD)	8
0.5.4 Data Flow Explanation	10
0.6 Security Architecture (INSA Compliance)	10
0.6.1 Network Security	11
0.6.2 Container Isolation	11
0.6.3 Secrets Management	11
0.6.4 Monitoring and Logging	11
0.6.5 Compliance	11
0.7 UML Diagrams	12
0.7.1 Use Case Diagram	12
0.7.2 Class Diagram	13
0.7.3 Sequence Diagram: Course Enrollment	15
0.7.4 Sequence Diagram: Course Creation (Educator)	16
0.8 Module Interaction	17
0.9 References	17
.1 Full Apache License 2.0 Text	18

List of Figures

1	System Architecture Diagram showing Caddy, LMS, CMS, MFE, databases, and supporting services.	7
2	Context DFD showing external entities (Learner, Educator, Administrator) and the system boundary.	8
3	Level 1 DFD showing main processes: Authentication, Course Management, Enrollment, Content Delivery, Assessment, and Reporting.	9
4	Use Case Diagram showing Educator and Learner interactions.	12
5	Core entities: User, Course, Enrollment, Assessment, Grade, Discussion. . .	14
6	Learner enrollment flow.	15
7	Educator course creation flow.	16

List of Tables

0.1 Document Management and Quality Assurance

0.1.1 Document Purpose

This System & Architecture Design Document (SDD) provides a comprehensive description of the Moove Education Platform's system architecture, including physical deployment, logical components, security architecture, and data flows. It serves as the primary reference for understanding how the system is built and how its components interact.

0.1.2 Diagram Management

All architectural and UML diagrams in this document are created using professional tools:

- **Primary Tool:** PlantUML with custom styling for consistent, professional output
- **Diagram Types:** System Architecture, DFD, Use Case, Class, Sequence diagrams
- **Source Management:** All diagram source (.puml) files are stored in the `plantuml/` directory
- **Build Integration:** Diagrams are automatically generated during the LaTeX build process
- **Quality Control:** Each diagram is reviewed for accuracy and completeness

0.1.3 Diagram Standards

- **System Architecture:** Uses C4 model style with clear component boundaries
- **DFD:** Follows Yourdon & Coad notation for consistency
- **Use Case:** UML 2.0 standard notation
- **Class Diagrams:** UML 2.0 with proper multiplicity notation
- **Sequence Diagrams:** UML 2.0 with proper lifeline notation

0.1.4 Quality Assurance

- All diagrams are validated against PlantUML syntax
- Diagrams are reviewed against actual system implementation
- Security architecture diagrams are validated against INSA requirements

0.2 System Overview

Moove Education Platform is an Open edX instance deployed using Tutor (v21.0.8) on Ubuntu 22.04. The architecture is container-based (Docker) and microservices-oriented. All components are orchestrated by Tutor and run inside Docker containers, with Caddy acting as the reverse proxy and TLS terminator.

0.3 Physical Architecture

The deployment consists of the following containers:

- **LMS (Learning Management System):** Learner-facing operations (courses, enrollments, assessments). Runs uWSGI.
- **CMS (Course Management System):** Educator authoring interface (Studio).
- **LMS Worker / CMS Worker:** Celery workers for background tasks.
- **MFE (Micro-Frontends):** Serves modern frontend applications via Caddy.
- **Caddy:** Reverse proxy and automatic TLS (ports 80/443).
- **MySQL 8.4:** Primary relational database.
- **MongoDB 7.0:** Course structure and discussion data.
- **Redis 7.4:** Caching and Celery broker.
- **Meilisearch 1.8:** Search engine.
- **SMTP (Exim):** Email relay.

0.4 Logical Architecture

0.4.1 Presentation Layer

- MFE apps (Account, Authn, Learner Dashboard) via Caddy. - Django templates for legacy parts.

0.4.2 Business Logic Layer

- Django/EdX-platform (Python) with plugins (Indigo theme, MFE). - Celery for asynchronous tasks.

0.4.3 Data Layer

- MySQL: Users, courses, grades, enrollments. - MongoDB: Course structures, discussions. - Redis: Session, cache, Celery broker. - Meilisearch: Search index.

0.5 System Architecture and Data Flow

0.5.1 Key Components

The Moove Education Platform consists of the following main components:

- **Reverse Proxy / TLS Termination:** Caddy handles all external HTTPS traffic, terminates TLS, and routes requests to the appropriate internal services.

- **Learning Management System (LMS):** Django application that serves learner-facing pages, handles enrollments, assessments, and grading.
- **Course Management System (CMS):** Django application (Studio) used by educators to author and manage courses.
- **Micro-Frontends (MFE):** React-based frontend applications (Account, Authn, Learner Dashboard) served via Caddy.
- **Background Workers:** Celery workers for asynchronous tasks (email, grading, analytics).
- **Primary Databases:** MySQL for user, course, and grade data; MongoDB for course structure and discussion data.
- **Caching and Messaging:** Redis for session storage, caching, and Celery broker.
- **Search Engine:** Meilisearch for full-text search across courses and content.
- **Email Relay:** SMTP (Exim) for sending system emails.

0.5.2 System Architecture Diagram

The following diagram illustrates the high-level system architecture, showing how components interact and the network segmentation used to isolate services.

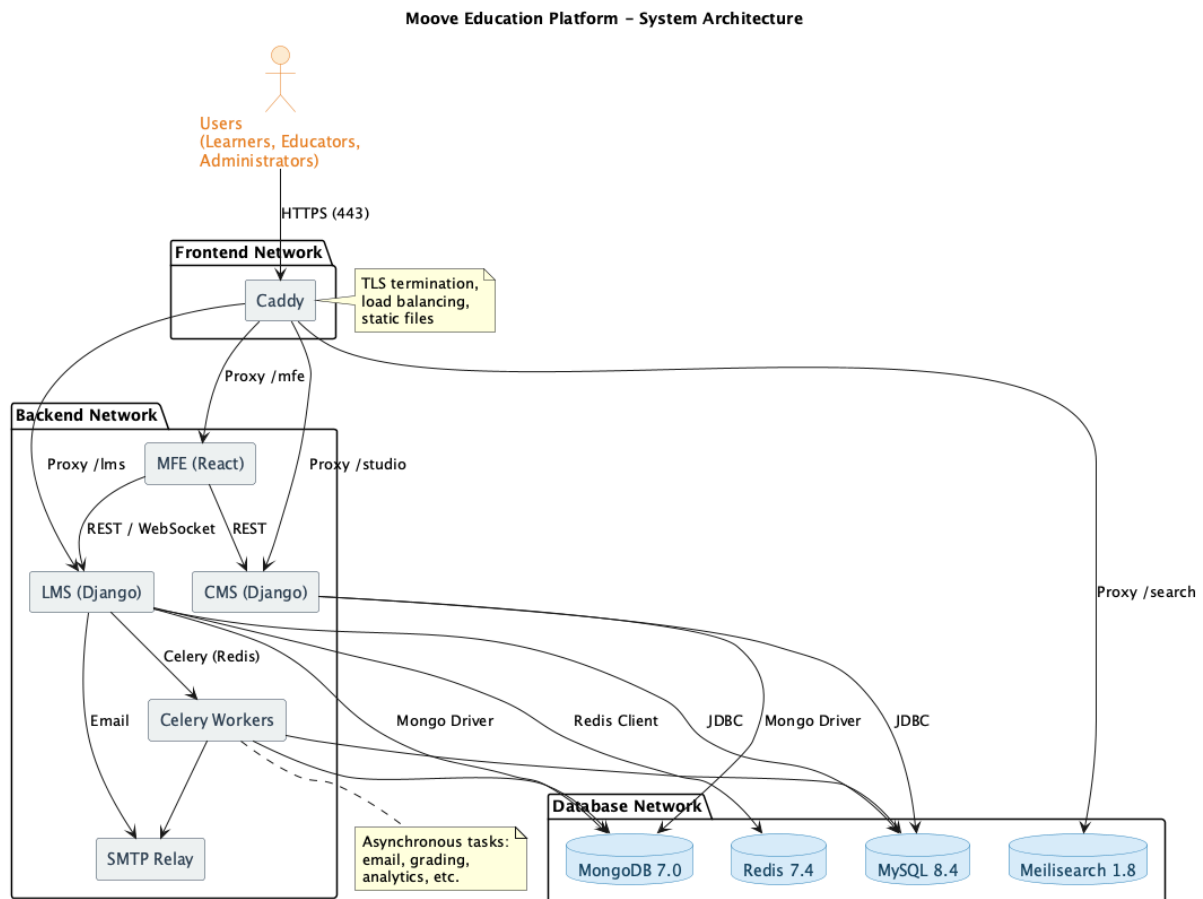


Figure 1: System Architecture Diagram showing Caddy, LMS, CMS, MFE, databases, and supporting services.

Explanation of the System Architecture Diagram

This diagram depicts the entire Moove Education Platform as a set of networked components grouped into three logical networks: **Frontend**, **Backend**, and **Database**.

- **External Users** (Learners, Educators, Administrators) connect to the system over the internet via **HTTPS**.
- **Caddy** acts as the single entry point (reverse proxy). It terminates TLS, routes requests to the appropriate service, and serves static assets. It also provides load balancing and automatic certificate renewal via Let's Encrypt.
- The **Frontend Network** contains only Caddy, which is exposed to the internet on ports 80 and 443.
- The **Backend Network** hosts the core application services:

- **LMS (Django):** handles all learner operations, including enrollment, content delivery, assessments, and grading.
- **CMS (Django):** the Studio interface for educators to create and manage courses.
- **MFE (React):** micro-frontend applications that provide modern user interfaces for authentication, account management, and learner dashboard.
- **Celery Workers:** process asynchronous tasks such as sending emails, grading background jobs, and updating analytics.

- **SMTP Relay (Exim)**: sends system emails (notifications, confirmations) to users.

- The **Database Network** is isolated from the backend and contains all persistent data stores:

- **MySQL 8.4**: stores relational data (users, courses, enrollments, grades, etc.).
- **MongoDB 7.0**: stores course structure (XBlock content) and discussion forum data.
- **Redis 7.4**: used for caching, session storage, and as a message broker for Celery.
- **Meilisearch 1.8**: provides fast full-text search over course content and discussions.

Connections between components are shown with arrows: - Caddy proxies requests to LMS, CMS, MFE, and Meilisearch. - LMS and CMS communicate with MySQL, MongoDB, and Redis. - LMS interacts with Celery (via Redis) and SMTP. - MFE communicates with LMS and CMS via REST APIs. - Workers access MySQL, MongoDB, and SMTP for background tasks.

This separation into networks enforces security boundaries: only Caddy is exposed externally; all other services are internal and accessible only within the Docker environment.

0.5.3 Data Flow Diagram (DFD)

The Data Flow Diagram shows how data moves through the system, from user interactions to storage and retrieval.

Context Diagram (Level 0)

The Context DFD shows the system boundary and external entities that interact with the platform.

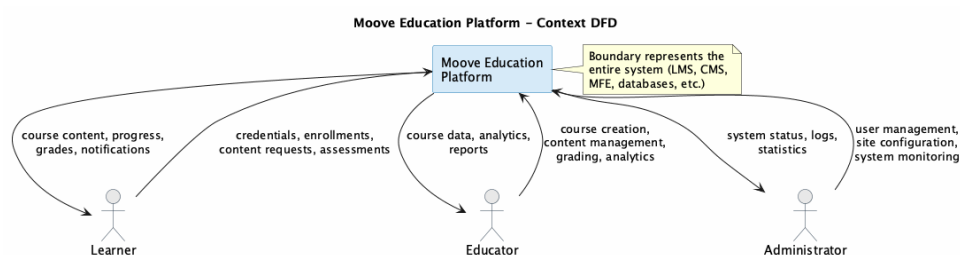


Figure 2: Context DFD showing external entities (Learner, Educator, Administrator) and the system boundary.

Explanation of the Context DFD

The Context Diagram (Level 0) presents the entire Moove Education Platform as a single process (the system boundary) and identifies the three primary external entities that interact with it:

- **Learner**: provides credentials, enrollment requests, content requests, and assessment submissions. Receives course content, progress data, grades, and notifications.

Educator: supplies course creation and management inputs, content updates, grading, and analytics queries. Receives course data, analytics reports, and system feedback. - **Administrator:** performs user management, site configuration, and system monitoring. Receives system status, logs, and statistics.

The data flows are labelled with the type of information exchanged. This diagram establishes the overall scope of the system and the interfaces with external users.

Level 1 DFD

The Level 1 DFD shows the main processes within the system and how they interact with data stores and external entities.

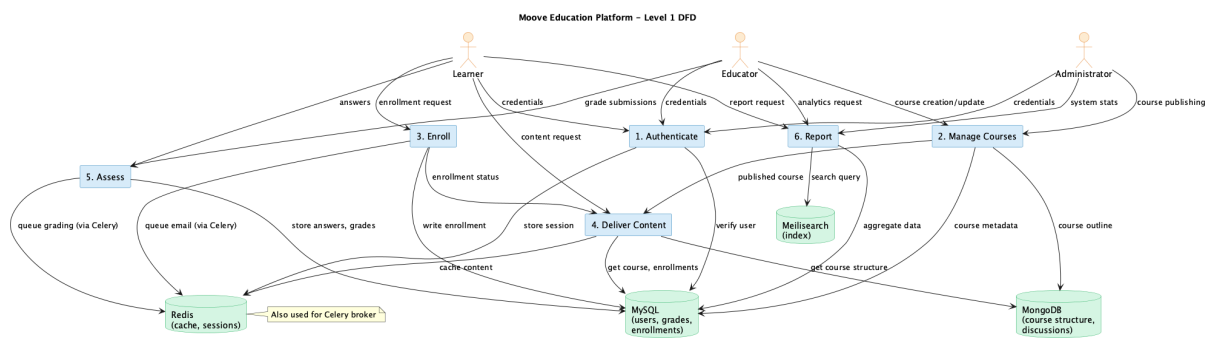


Figure 3: Level 1 DFD showing main processes: Authentication, Course Management, Enrollment, Content Delivery, Assessment, and Reporting.

Explanation of the Level 1 DFD

This diagram decomposes the system into six core processes and three data stores:

Processes:

- 1. Authenticate** – validates user credentials (learner, educator, admin). Interacts with MySQL (user table) and Redis (session storage).
- 2. Manage Courses** – handles course creation, updates, and publishing. Writes course metadata to MySQL and course structure (outline) to MongoDB.
- 3. Enroll** – manages learner enrollment. Writes enrollment records to MySQL and triggers a Celery task (via Redis) to send a confirmation email.
- 4. Deliver Content** – retrieves course content based on learner requests. Reads course metadata from MySQL and course structure from MongoDB; uses Redis for caching.
- 5. Assess** – processes learner answers, stores grades in MySQL, and queues grading tasks via Redis.
- 6. Report** – generates analytics and search results. Queries MySQL for aggregate data and uses Meilisearch for full-text search.

Data Stores:

- **MySQL:** persistent storage for users, courses, grades, enrollments, and other relational data.
- **MongoDB:** stores course structure (hierarchical content) and discussion data.
- **Redis:** serves as a cache, session store, and Celery broker for asynchronous tasks.

Data flows between processes and stores are labelled with the type of operation (read/write). The diagram also shows that the Enroll process communicates with Deliver (enrollment status affects content visibility) and Manage Courses feeds published courses to Deliver.

0.5.4 Data Flow Explanation

The system processes data as follows:

- **Authentication:** User credentials are submitted via the MFE or LMS. Caddy passes the request to the LMS, which validates against MySQL and returns a session token (JWT).
- **Course Management:** Educators create course structures in the CMS. Data is stored in MySQL (course metadata) and MongoDB (course outline, XBlock content). When published, the LMS reads this data to render course content.
- **Enrollment:** When a learner enrolls, the LMS writes the enrollment record to MySQL and triggers a Celery task to send a confirmation email.
- **Content Delivery:** Learners request course pages; the LMS fetches the course structure from MongoDB, renders the content (using Django templates or MFE), and serves it via Caddy.
- **Assessments:** Learner answers are sent to the LMS, validated, and stored in MySQL. Grade calculations are performed asynchronously by Celery workers.
- **Search:** Meiliseach indexes course content and discussions. Search queries are routed through Caddy to the Meiliseach API.
- **Notifications:** The system generates notification events (e.g., discussion replies, assignment due dates) via Redis and Celery, sending emails through the SMTP relay.

0.6 Security Architecture (INSA Compliance)

This section describes the security controls implemented to meet INSA requirements. Detailed risk assessment and threat modelling are provided in the *Security Architecture & Threat Model* document.

0.6.1 Network Security

- Host firewall (UFW) is enabled and configured to allow only SSH (22), HTTP (80), and HTTPS (443) from external networks.
- Internal container networking is segmented into three Docker networks: **frontend**, **backend**, and **database**. The database network is internal and not exposed to the host.
- All unnecessary ports and services are disabled.
- Rate limiting and brute-force protection are provided by fail2ban.

0.6.2 Container Isolation

- Each service runs in its own container with minimal privileges.
- Tutor runs under a dedicated non-root user (**tutor**) with membership in the **docker** group.
- Resource limits (CPU, memory) are applied to containers to prevent resource exhaustion.

0.6.3 Secrets Management

- All sensitive configuration values (passwords, API keys, private keys) are stored encrypted and rotated regularly.
- Secrets are managed via Tutor's configuration system and not hard-coded in source code.
- JWT RSA private key is stored securely and rotated periodically.

0.6.4 Monitoring and Logging

- Centralised logging is implemented using Tutor log analysis.
- Security events (login failures, privilege changes) are logged and monitored.
- Weekly vulnerability scanning of container images is performed using Trivy.

0.6.5 Compliance

- The architecture is designed to meet CMCSRS requirements for access control, cryptography, and incident response.
- Annual security audits and accreditation are planned.
- All security controls are documented and tested.

For a complete threat model and risk assessment, refer to the *Security Architecture & Threat Model* (Security_OpenEdX.tex).

0.7 UML Diagrams

0.7.1 Use Case Diagram

The Use Case Diagram shows the main interactions between actors and the system.

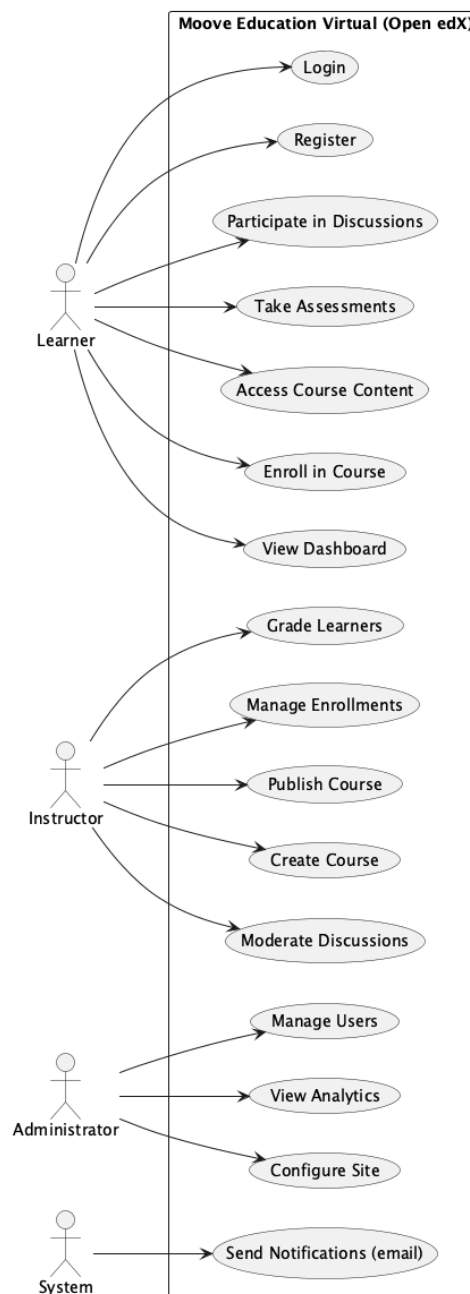


Figure 4: Use Case Diagram showing Educator and Learner interactions.

Explanation of the Use Case Diagram

This UML Use Case Diagram identifies the primary actors and their interactions with the Moove Education Platform.

- **Actors:**

- **Learner:** an end-user who consumes educational content.
- **Instructor:** a course creator and manager.
- **Administrator:** a system operator with full control.
- **System:** an automated component that sends notifications.

- Use Cases (grouped by actor):

- **Learner:** Register, Login, View Dashboard, Enroll in Course, Access Course Content, Take Assessments, Participate in Discussions.
- **Instructor:** Create Course, Publish Course, Manage Enrollments, Grade Learners, Moderate Discussions.
- **Administrator:** Manage Users, Configure Site, View Analytics.
- **System:** Send Notifications (email, triggered by various events).

The diagram illustrates that learners interact primarily with content consumption and assessment, while instructors manage course creation and student progress. Administrators handle platform governance. The System use case represents automated background processes.

0.7.2 Class Diagram

The Class Diagram shows the core entities and their relationships.

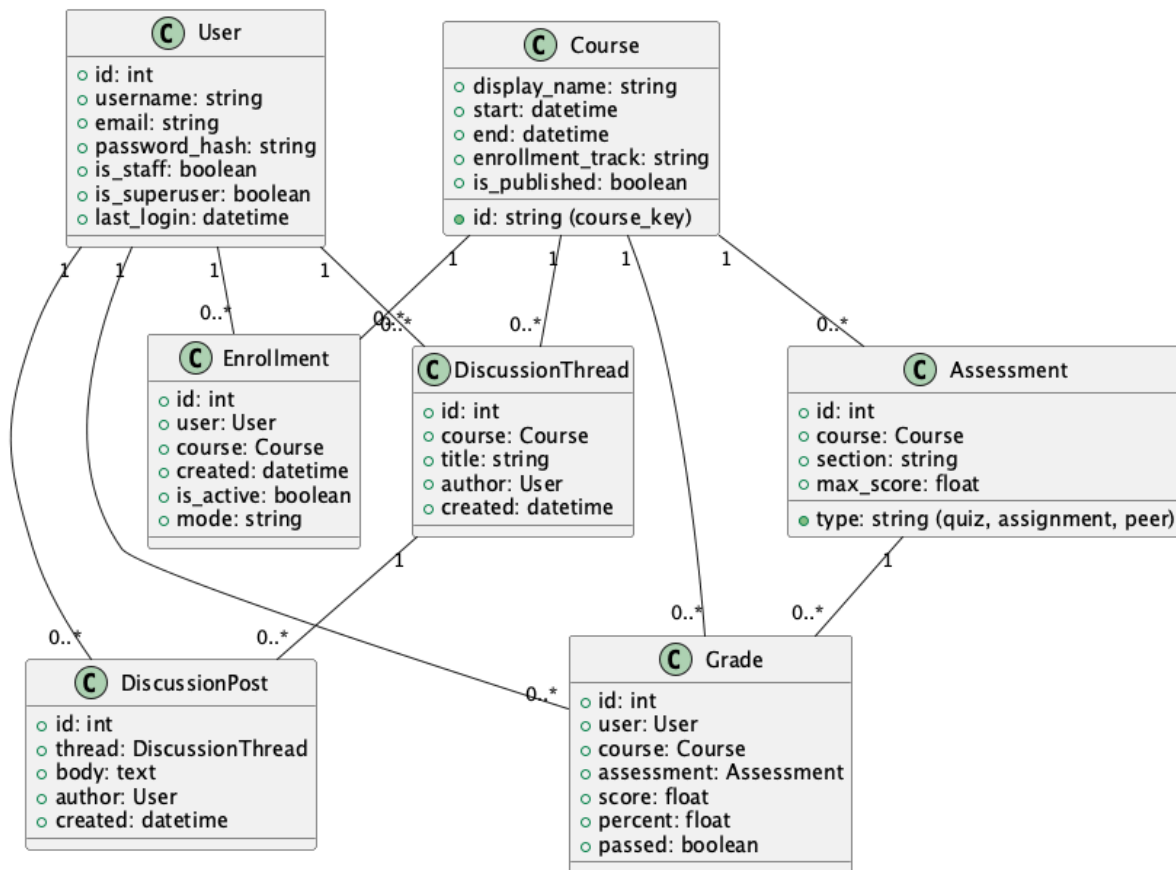


Figure 5: Core entities: User, Course, Enrollment, Assessment, Grade, Discussion.

Explanation of the Class Diagram

This diagram defines the main domain classes and their associations, which map to the database tables.

- **User**: represents a platform user (learner, instructor, admin). Attributes include `id`, `username`, `email`, `password_hash`, `is_staff`, `is_superuser`, `last_login`.

- **Course**: a course offering. Attributes: `id` (course key), `display_name`, `start`, `end`, `enrollment_track`, `is_published`.

- **Enrollment**: links a User to a Course. Attributes: `created`, `is_active`, `mode` (audit/verified). Multiplicity: a User can have many Enrollments; a Course can have many Enrollments.

- **Assessment**: a learning activity (quiz, assignment, peer review). Attributes: `section`, `type`, `max_score`. Each Course has multiple Assessments.

- **Grade**: a score for a User on a specific Assessment in a Course. Attributes: `score`, `percent`, `passed`. Links User, Course, and Assessment.

- **DiscussionThread**: a forum topic within a Course. Attributes: `title`, `created`. Each Course has many Threads; a User can author many Threads.

- **DiscussionPost**: a reply within a Thread. Attributes: `body`, `created`. Each Thread has many Posts; a User can author many Posts.

The relationships are shown with multiplicities (e.g., 1-*, 0-*). This class model underpins the relational database schema and the object-relational mapping (Django ORM).

0.7.3 Sequence Diagram: Course Enrollment

The Sequence Diagram shows the flow of a learner enrollment process.

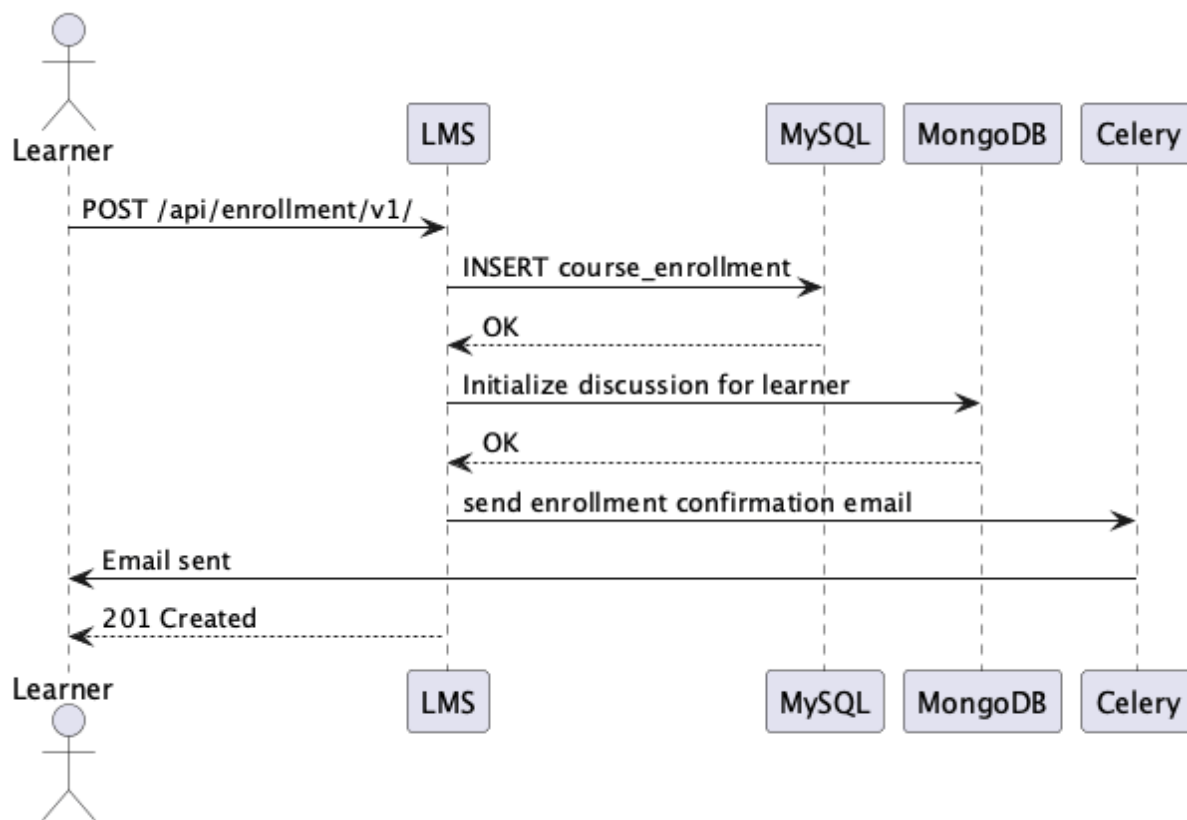


Figure 6: Learner enrollment flow.

Explanation of the Enrollment Sequence Diagram

This diagram illustrates the step-by-step interaction for a learner enrolling in a course.

1. The **Learner** sends a `POST /api/enrollment/v1/` request to the **LMS**.
2. The LMS inserts a new record into the **MySQL** database (`course_enrollment` table). MySQL confirms success.
3. The LMS then initialises discussion data for the learner in **MongoDB** (e.g., creating a discussion topic if needed). MongoDB confirms.
4. The LMS queues an asynchronous task to send a confirmation email by pushing a message to **Celery** (via Redis).
5. The Celery worker processes the task and sends an email to the Learner.
6. Finally, the LMS returns a `201 Created` response to the Learner.

This sequence shows synchronous operations (database writes) and asynchronous tasks (email) to maintain responsiveness.

0.7.4 Sequence Diagram: Course Creation (Educator)

The Sequence Diagram shows the flow of an educator creating a course.

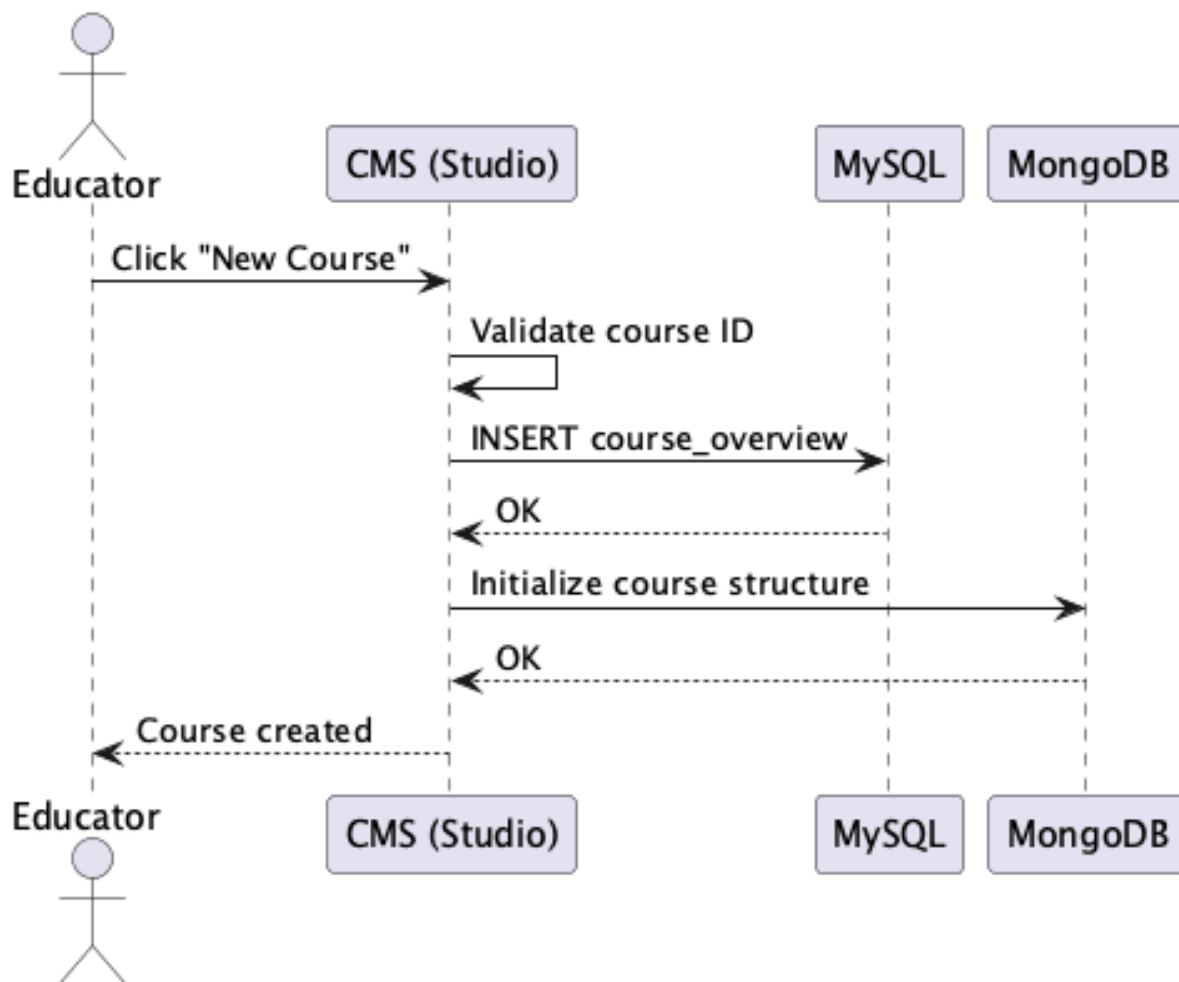


Figure 7: Educator course creation flow.

Explanation of the Course Creation Sequence Diagram

This diagram details the process of creating a new course from the CMS.

1. The **Educator** clicks "New Course" in the Studio UI.
2. The **CMS** validates the proposed course ID (ensuring uniqueness).
3. The CMS inserts a new row into the **MySQL** `course_overview` table with course metadata. MySQL returns success.
4. The CMS initialises the course structure in **MongoDB** (e.g., creates the root node for the course). MongoDB confirms.
5. The CMS returns a confirmation to the Educator that the course skeleton has been created.

This sequence is synchronous; the course is immediately ready for further content editing.

0.8 Module Interaction

Refer to the TDD for detailed component diagrams and API interactions.

0.9 References

- *Security Architecture & Threat Model – Moove Education Platform* (Security_OpenEdX.tex)
- CMCSRS v2.0
- FDRE National Cybersecurity Policy (2024)
- Computer Crime Proclamation No. 958/2016
- Electronic Signature Proclamation No. 1072/2018

Apache License 2.0 Attribution

This product includes software developed by the **Open edX Project** (<https://open.edx.org/>) licensed under the Apache License, Version 2.0 (<http://www.apache.org/licenses/LICENSE-2.0>).

Moove Education Platform is a derivative work of Open edX. No modifications have been made to the original license terms. A copy of the full license text is available in the root of the source repository and in Appendix A of this document.

.1 Full Apache License 2.0 Text